CPU Scheduling Comparison Project (RR vs SJF)

Introduction:

This project compares two CPU scheduling algorithms: Round Robin (RR) and Shortest Job First (SJF).
The goal is to analyze how each algorithm performs in terms of waiting time, response time, turnaround time, and fairness.

Objectives:

. Compare Round Robin and SJF scheduling algorithms

. Measure performance using WT, TAT, RT

. Study fairness vs efficiency trade-off

. Analyze effect of time quantum in Round Robin

. Evaluate best algorithm for different workloads

Algorithms Explanation:

Round Robin (RR)

Round Robin is a time-sharing scheduling algorithm where each process is assigned a fixed time quantum.
Processes are executed in a circular queue, ensuring fairness.

SJF (Shortest Job First)

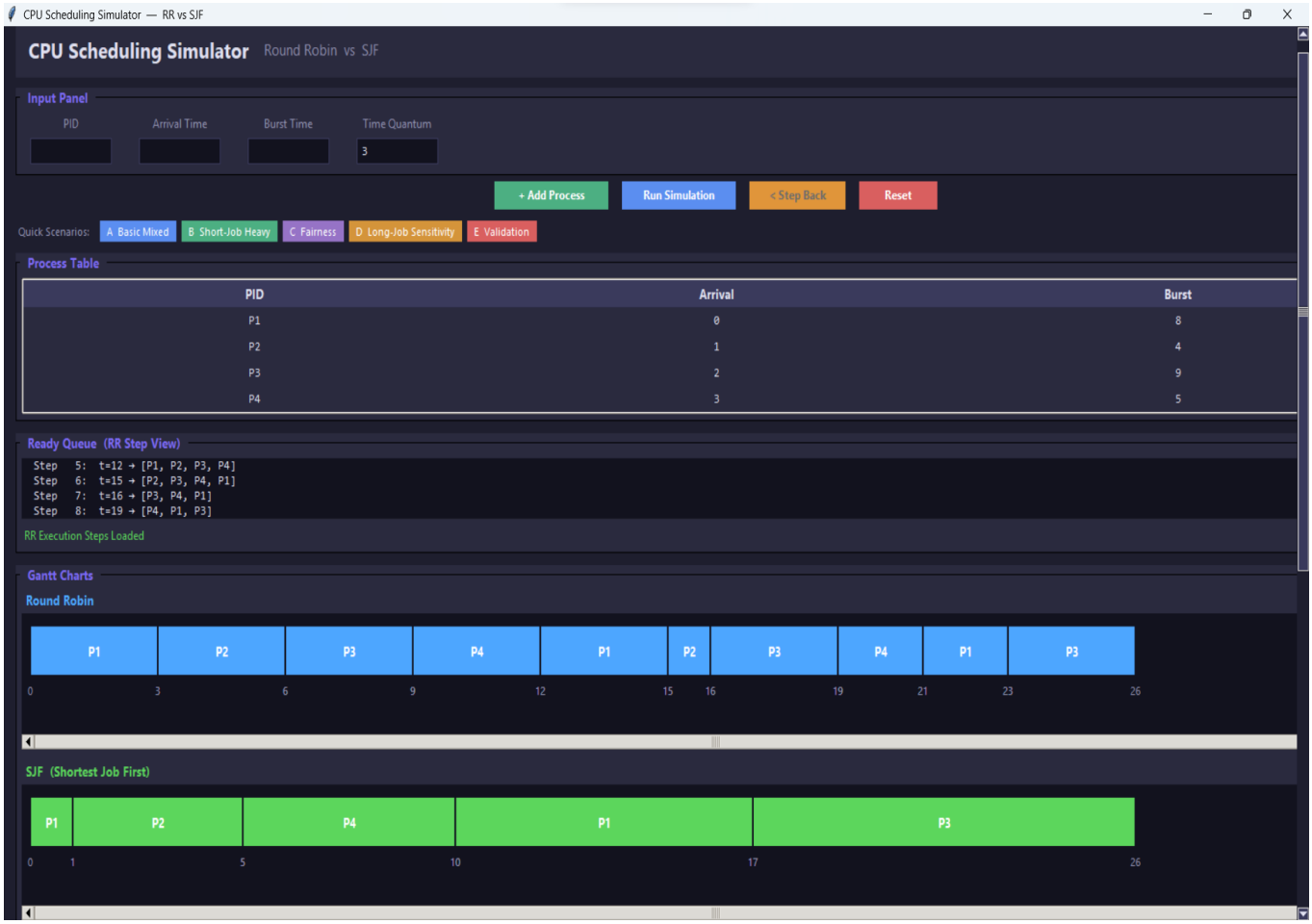
SJF selects the process with the smallest burst time first. It is efficient in reducing waiting time but may cause starvation for long processes.

Test Scenarios

- A. Basic Mixed Workload
- B. Short Job Heavy Case
- C. Fairness Case
- D. Long Job Sensitivity Case
- E. Validation Case

Screenshots Section (Placeholders)

A. Basic Mixed Workload Screenshot Placeholder



RR Metrics

PID	WT	TAT	RT
P1	15	23	0
P2	11	15	2
P3	15	24	4
P4	13	18	6

SJF Metrics

PID	WT	TAT
P1	9	17
P2	0	4
P3	15	24
P4	2	7

Average Metrics Comparison

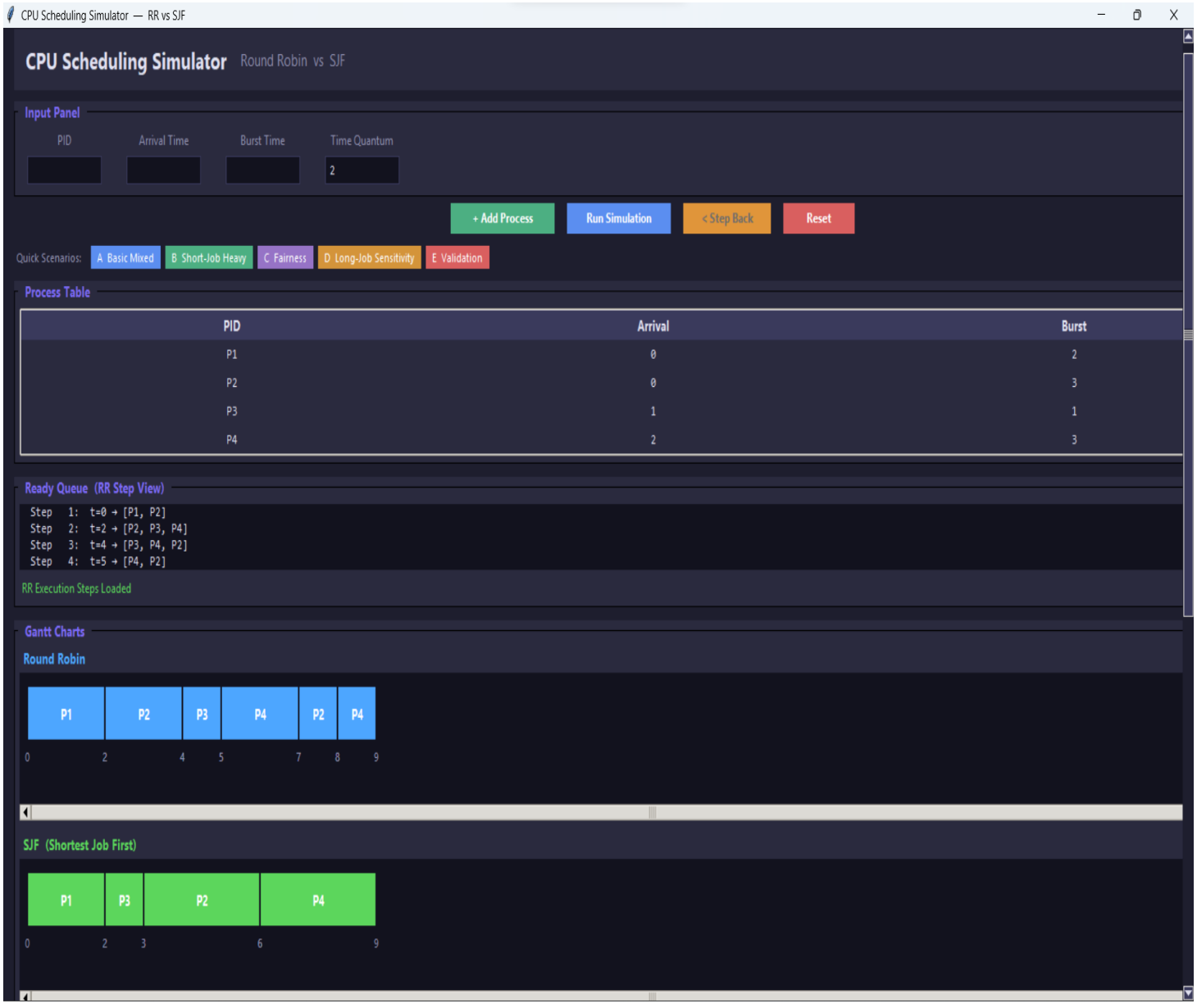
Algorithm	Avg WT	Avg TAT	Avg RT
Round Robin	13.5	20.0	3.0
SJF	6.5	13.0	4.25

Final Analysis

```
================ FINAL ANALYSIS ================

Average Metrics:
  WT   ->  RR = 13.50   |   SJF = 6.50
  TAT  ->  RR = 20.00   |   SJF = 13.00
  RT   ->  RR = 3.00    |   SJF = 4.25

---------- Comparison ----------

  Fairness vs Efficiency:
    RR  -> fair time-sharing; every process gets CPU turns.
    SJF -> efficient; shortest jobs finish first.

  CPU Distribution:
```


```
================ FINAL ANALYSIS ================

Average Metrics:
  WT   ->  RR = 13.50   |   SJF = 6.50
  TAT  ->  RR = 20.00   |   SJF = 13.00
  RT   ->  RR = 3.00    |   SJF = 4.25

---------- Comparison ----------

  Fairness vs Efficiency:
    RR  -> fair time-sharing; every process gets CPU turns.
    SJF -> efficient; shortest jobs finish first.

  CPU Distribution:
    RR  -> rotates using quantum = 3.
    SJF -> always picks the shortest available job.

  Quantum Effect (RR):
    Small quantum -> better responsiveness, more context switches.
    Large quantum -> behaves like FCFS.

  Response Time:
    RR  -> generally better first-response time.
    SJF -> may delay long processes significantly.

  Starvation Risk:
    RR  -> none; long jobs always make progress.
    SJF -> possible if short jobs keep arriving.

---------- Result ----------

  Better Avg WT  -> SJF
  Better Avg TAT -> SJF
  Better Avg RT  -> Round Robin

  Quantum = 3

================================================
```

B. Short Job Heavy Case Screenshot Placeholder



RR Metrics

PID	WT	TAT	RT
P1	0	2	0
P2	5	8	2
P3	3	4	3
P4	4	7	3

SJF Metrics

PID	WT	TAT
P1	0	2
P2	3	6
P3	1	2
P4	4	7

Average Metrics Comparison

Algorithm	Avg WT	Avg TAT	Avg RT
Round Robin	3.0	5.25	2.0
SJF	2.0	4.25	2.0

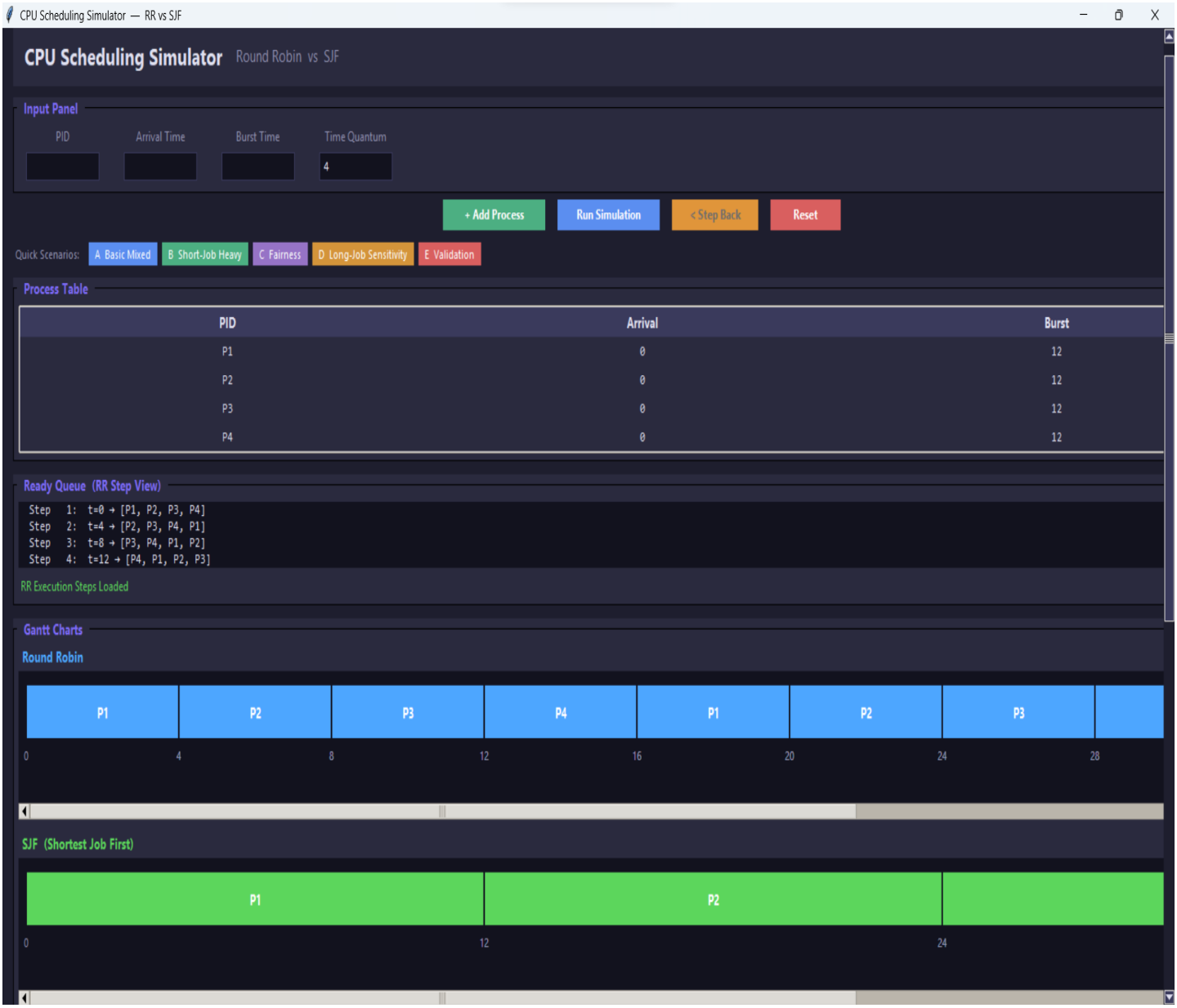
Final Analysis

```
================ FINAL ANALYSIS ================

Average Metrics:
  WT   ->  RR = 3.00   |   SJF = 2.00
  TAT  ->  RR = 5.25   |   SJF = 4.25
  RT   ->  RR = 2.00   |   SJF = 2.00

---------- Comparison ----------

  Fairness vs Efficiency:
    RR  -> fair time-sharing; every process gets CPU turns.
    SJF -> efficient; shortest jobs finish first.

  CPU Distribution:
```


```
================ FINAL ANALYSIS ================

Average Metrics:
  WT   ->  RR = 3.00   |   SJF = 2.00
  TAT  ->  RR = 5.25   |   SJF = 4.25
  RT   ->  RR = 2.00   |   SJF = 2.00

---------- Comparison ----------

  Fairness vs Efficiency:
    RR  -> fair time-sharing; every process gets CPU turns.
    SJF -> efficient; shortest jobs finish first.

  CPU Distribution:
```


```
    RR  -> rotates using quantum = 2.
    SJF -> always picks the shortest available job.

  Quantum Effect (RR):
    Small quantum -> better responsiveness, more context
switches.
    Large quantum -> behaves like FCFS.

  Response Time:
    RR  -> generally better first-response time.
    SJF -> may delay long processes significantly.

  Starvation Risk:
    RR  -> none; long jobs always make progress.
    SJF -> possible if short jobs keep arriving.

---------- Result ----------

  Better Avg WT  -> SJF
  Better Avg TAT -> SJF
  Better Avg RT  -> SJF

  Quantum = 2

================================================
```

C. Fairness Case Screenshot Placeholder



RR Metrics

PID	WT	TAT	RT
P1	24	36	0
P2	28	40	4
P3	32	44	8
P4	36	48	12

SJF Metrics

PID	WT	TAT
P1	0	12
P2	12	24
P3	24	36
P4	36	48

Average Metrics Comparison

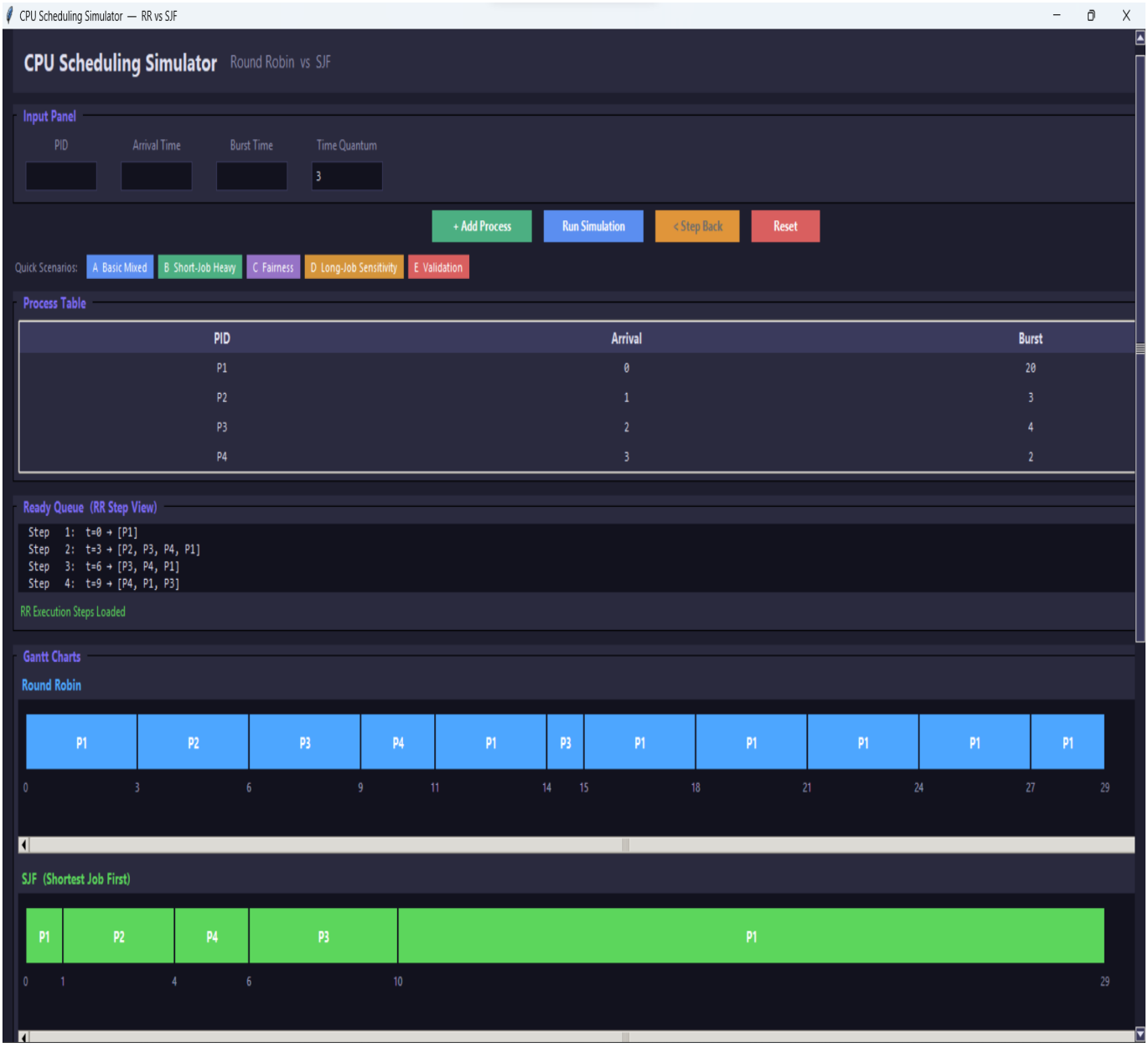
Algorithm	Avg WT	Avg TAT	Avg RT
Round Robin	30.0	42.0	6.0
SJF	18.0	30.0	18.0

Final Analysis

```
=============== FINAL ANALYSIS ===============

Average Metrics:
  WT   ->  RR = 30.00   |   SJF = 18.00
  TAT  ->  RR = 42.00   |   SJF = 30.00
  RT   ->  RR = 6.00    |   SJF = 18.00

--------------- Comparison ---------------

  Fairness vs Efficiency:
    RR  -> fair time-sharing; every process gets CPU turns.
    SJF -> efficient; shortest jobs finish first.

  CPU Distribution:
```


```
================ FINAL ANALYSIS ================

Average Metrics:
 WT  -> RR = 30.00 | SJF = 18.00
 TAT -> RR = 42.00 | SJF = 30.00
 RT  -> RR = 6.00 | SJF = 18.00

---------- Comparison ----------

 Fairness vs Efficiency:
  RR -> fair time-sharing; every process gets CPU turns.
  SJF -> efficient; shortest jobs finish first.

 CPU Distribution:
  RR -> rotates using quantum = 4.
  SJF -> always picks the shortest available job.

 Quantum Effect (RR):
  Small quantum -> better responsiveness, more context switches.
  Large quantum -> behaves like FCFS.

 Response Time:
  RR -> generally better first-response time.
  SJF -> may delay long processes significantly.

 Starvation Risk:
  RR -> none; long jobs always make progress.
  SJF -> possible if short jobs keep arriving.

---------- Result ----------

 Better Avg WT  -> SJF
 Better Avg TAT -> SJF
 Better Avg RT  -> Round Robin

 Quantum = 4

================================================
```

D. Long Job Sensitivity Case Screenshot Placeholder



RR Metrics

PID	WT	TAT	RT
P1	9	29	0
P2	2	5	2
P3	9	13	4
P4	6	8	6

SJF Metrics

PID	WT	TAT
P1	9	29
P2	0	3
P3	4	8
P4	1	3

Average Metrics Comparison

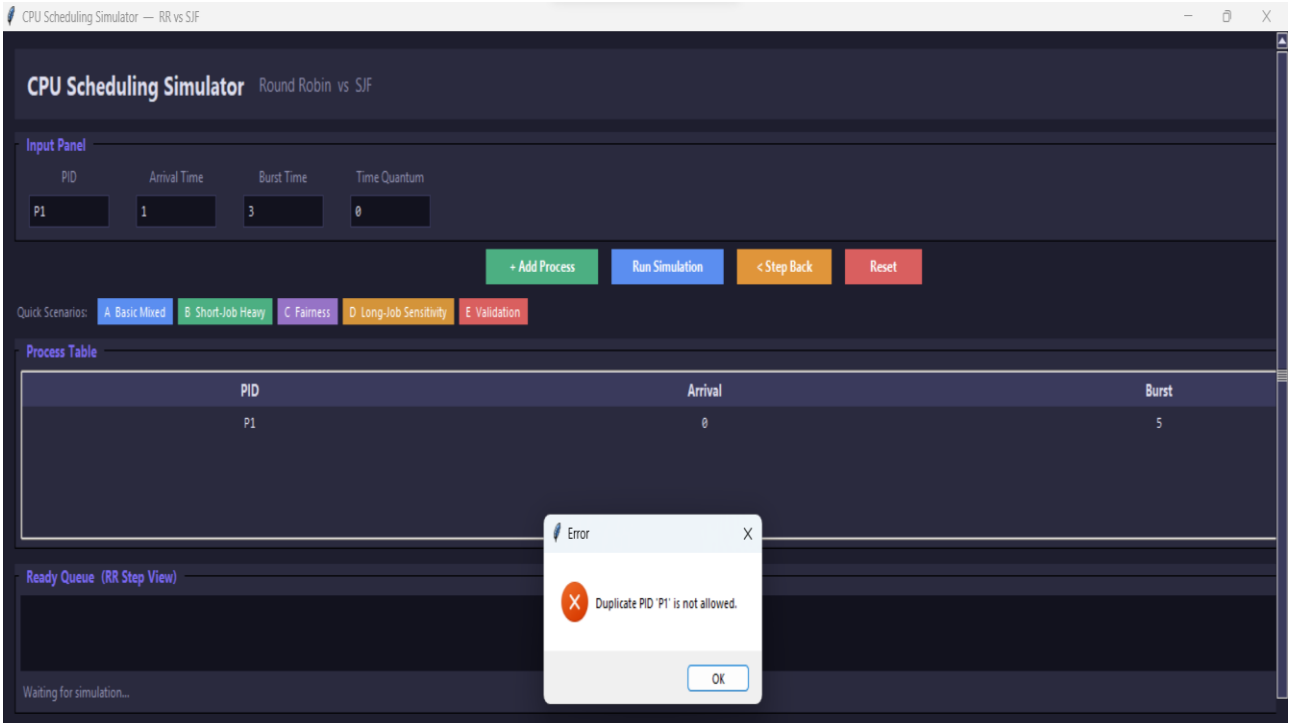
Algorithm	Avg WT	Avg TAT	Avg RT
Round Robin	6.5	13.75	3.0
SJF	3.5	10.75	1.25

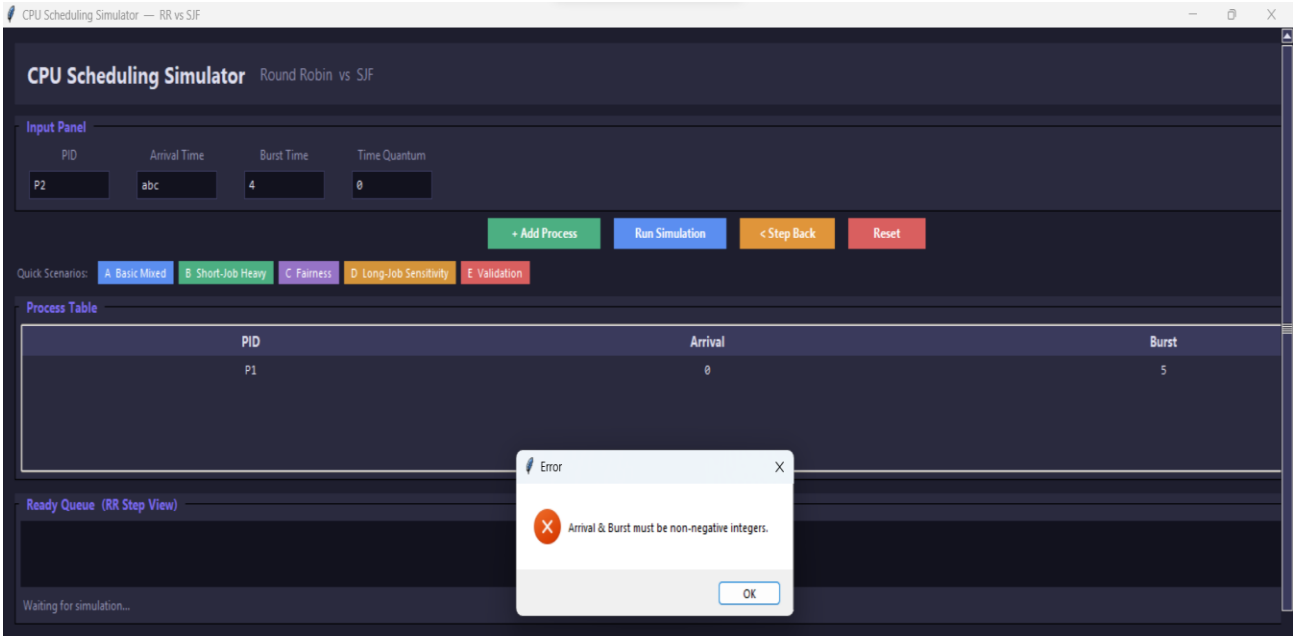
Final Analysis

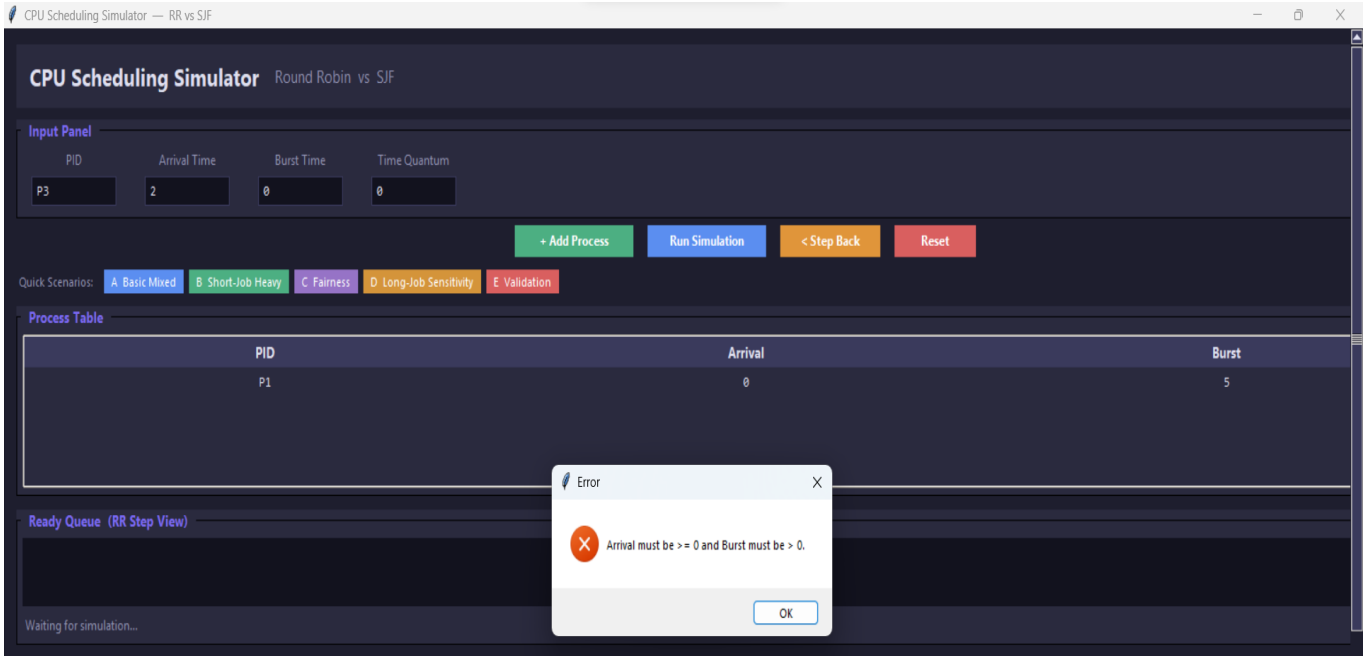
```
=============== FINAL ANALYSIS ===============

Average Metrics:
  WT   ->  RR = 6.50   |   SJF = 3.50
  TAT  ->  RR = 13.75   |   SJF = 10.75
  RT   ->  RR = 3.00   |   SJF = 1.25

---------- Comparison ----------

  Fairness vs Efficiency:
    RR  -> fair time-sharing; every process gets CPU turns.
    SJF -> efficient; shortest jobs finish first.

  CPU Distribution:
```

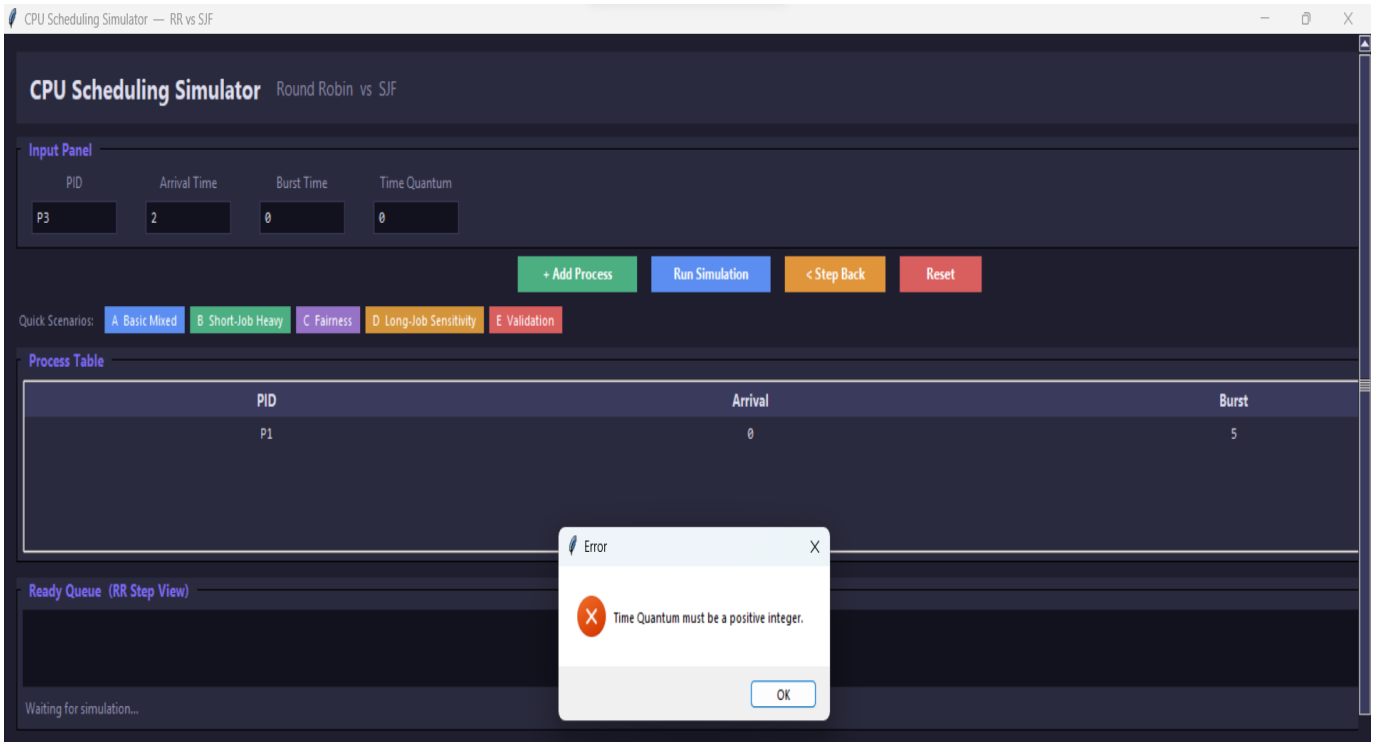
```
================ FINAL ANALYSIS ================

Average Metrics:
  WT   ->  RR = 6.50   |   SJF = 3.50
  TAT  ->  RR = 13.75   |   SJF = 10.75
  RT   ->  RR = 3.00   |   SJF = 1.25

---------- Comparison ----------

  Fairness vs Efficiency:
    RR  -> fair time-sharing; every process gets CPU turns.
    SJF -> efficient; shortest jobs finish first.

  CPU Distribution:
    RR  -> rotates using quantum = 3.
    SJF -> always picks the shortest available job.

  Quantum Effect (RR):
    Small quantum -> better responsiveness, more context switches.
    Large quantum -> behaves like FCFS.

  Response Time:
    RR  -> generally better first-response time.
    SJF -> may delay long processes significantly.

  Starvation Risk:
    RR  -> none; long jobs always make progress.
    SJF -> possible if short jobs keep arriving.

---------- Result ----------

  Better Avg WT  -> SJF
  Better Avg TAT  -> SJF
  Better Avg RT  -> SJF

  Quantum = 3

================================================
```

E. Validation Case Screenshot Placeholder






CPU Scheduling Simulator — RR vs SJF
CPU Scheduling Simulator
Round Robin vs SJF
Input Panel
PID
Arrival Time
Burst Time
Time Quantum
P3
2
0
0
+ Add Process
Run Simulation
< Step Back
Reset
Quick Scenarios:
A Basic Mixed
B Short-Job Heavy
C Fairness
D Long-Job Sensitivity
E Validation
Process Table
PID
Arrival
Burst
P1
0
5
Error
Arrival must be >= 0 and Burst must be > 0.
OK
Ready Queue (RR Step View)
Waiting for simulation...


CPU Scheduling Simulator — RR vs SJF
CPU Scheduling Simulator
Round Robin vs SJF
Input Panel
PID
Arrival Time
Burst Time
Time Quantum
P3
2
0
0
+ Add Process
Run Simulation
< Step Back
Reset
Quick Scenarios:
A Basic Mixed
B Short-Job Heavy
C Fairness
D Long-Job Sensitivity
E Validation
Process Table
PID
Arrival
Burst
P1
0
5
Error
Time Quantum must be a positive integer.
OK
Ready Queue (RR Step View)
Waiting for simulation...

Analysis Questions

1.Which algorithm gave lower average waiting time?

Shortest Job First (SJF) gave the lower average waiting time.
This is because SJF always selects the process with the shortest burst time first, which reduces the overall waiting time for shorter processes.

---

2. Which algorithm gave lower average response time?

Round Robin (RR) generally provided a better (lower) response time.
Since RR gives each process a time slice immediately, all processes start execution earlier compared to SJF.

---

3. Did Round Robin appear fairer across all processes?

Yes, **Round Robin appeared fairer**.
It distributes CPU time equally among all processes using time slicing, ensuring that no process is starved.

---

4. Did SJF complete short jobs more efficiently?

Yes, **SJF completed short jobs more efficiently**.
It prioritizes processes with the smallest burst time, allowing short jobs to finish very quickly.

---

5. How did the chosen quantum affect Round Robin behavior?

The **time quantum** significantly affected Round Robin performance:

- **Small quantum:** More fairness and better responsiveness, but higher context switching overhead.
- **Large quantum:** Fewer context switches and better efficiency, but RR starts behaving like FCFS and becomes less fair.

Overall, the selected quantum created a balance between responsiveness and system overhead.

6. Which algorithm would you recommend for the tested workload, and why?

- If the goal is **minimum waiting time and efficiency**, then **SJF is better**.
- If the goal is **fairness and responsiveness**, then **Round Robin is better**.

Recommendation:
For general workloads with mixed processes, **Round Robin is usually more practical**, while **SJF is better when short jobs dominate and burst times are known in advance**.

Conclusion

State which algorithm performed better on each major metric

- **Average Waiting Time:** SJF performed better
- **Response Time:** Round Robin performed better
- **Fairness:** Round Robin performed better
- **Short Job Completion:** SJF performed better

• State whether Round Robin appeared more balanced

Yes, **Round Robin appeared more balanced** because it distributes CPU time equally among all processes, preventing starvation and ensuring fairness.

• State whether SJF appeared more efficient

Yes, **SJF appeared more efficient**, especially in reducing average waiting time and completing short processes quickly.

• Explain the observed effect of the selected quantum

The **time quantum directly influenced Round Robin behavior**:

- A smaller quantum improved fairness and responsiveness but increased overhead due to frequent context switching.
- A larger quantum reduced overhead but decreased fairness and made the algorithm closer to First-Come First-Served (FCFS).

Therefore, the chosen quantum determined the trade-off between system efficiency and responsiveness.